```sql
-- ============================================================================
-- QUESTION 1: Customer Lifetime Value Ranking
-- Rank all customers by total profit generated using RANK() and DENSE_RANK()
-- side by side to demonstrate correct tie-handling behavior.
-- ============================================================================
select
    c.customer_name,
    sum(oi.profit) total_profit,
    rank() over (
        order by sum(oi.profit) desc
    ) profit_rank,
    dense_rank() over (
        order by sum(oi.profit) desc
    ) profit_dense_rank
from order_items oi
join orders o
    on o.order_id = oi.order_id
join customers c
    on c.customer_id = o.customer_id
group by c.customer_name
order by total_profit desc
limit 5;
-- INSIGHT: Emily Phan is the top customer by profit ($34,005.44), nearly
-- double customer #5. Retaining top-tier customers matters more than
-- acquiring new ones for protecting overall profitability.
```

customers 1

select c.customer_name, s | Enter a SQL expression to filter resul

Grid

Text

	customer_name	total_profit	profit_rank	profit_dense_rank
1	Emily Phan	34,005.44	1	1
2	Deborah Brumfield	31,121.22	2	2
3	Grant Carroll	27,977.29	3	3
4	Karen Carlisle	21,731.53	4	4
5	Alejandro Grove	20,588.92	5	5

```sql
-- ================================================================================
-- QUESTION 2: Running Total & Moving Average of Monthly Revenue
-- Calculate cumulative revenue over time alongside a 3-month moving average
-- to smooth out volatility and reveal the underlying trend.
-- ================================================================================
with monthly_revenue as (
    select
        o.year,
        o.month,
        o.month_name,
        sum(oi.sales) as revenue
    from order_items oi
    join orders o
        on o.order_id = oi.order_id
    group by o.year, o.month, o.month_name
)
select
    year,
    month,
    month_name,
    revenue,
    sum(revenue) over (
        order by year, month
    ) as running_total_revenue,
    round(
        avg(revenue) over (
            order by year, month
            rows between 2 preceding and current row
        ), 2
    ) as moving_avg_3month
from monthly_revenue
order by year, month;
-- Insight: Revenue accumulated to $14.92M by end of 2012, with the 3-month
-- moving average staying fairly flat ($300K-$330K range) in recent months
-- consistent with your earlier finding of no clear growth trend across 2009-2012
```

orders 1

with monthly_revenue as (| Enter a SQL expression to filter results (use

Grid | Text | Record

	123 year	123 mo	A-Z month_n	123 rever	123 running_tot	123 moving_a
1	2,009	1	January	516,302.97	516,302.97	516,302.97
2	2,009	2	February	332,480.63	848,783.6	424,391.8
3	2,009	3	March	411,628.71	1,260,412.31	420,137.44
4	2,009	4	April	393,276.47	1,653,688.78	379,128.6
5	2,009	5	May	230,145.54	1,883,834.32	345,016.91
6	2,009	6	June	263,456.06	2,147,290.38	295,626.02
7	2,009	7	July	380,503.97	2,527,794.35	291,368.52
8	2,009	8	August	329,754.74	2,857,549.09	324,571.59
9	2,009	9	September	325,292.3	3,182,841.39	345,183.67
10	2,009	10	October	361,555.28	3,544,396.67	338,867.44
11	2,009	11	November	248,933.45	3,793,330.12	311,927.01
12	2,009	12	December	415,809.35	4,209,139.47	342,099.36
13	2,010	1	January	336,526.68	4,545,666.15	333,756.49
14	2,010	2	February	271,580.52	4,817,246.67	341,305.52
15	2,010	3	March	217,808.01	5,035,054.68	275,305.07
16	2,010	4	April	266,968.6	5,302,023.28	252,119.04
17	2,010	5	May	283,534.31	5,585,557.59	256,103.64
18	2,010	6	June	293,080.67	5,878,638.26	281,194.53
19	2,010	7	July	229,885.5	6,108,523.76	268,833.49
20	2,010	8	August	207,937.01	6,316,460.77	243,634.39
21	2,010	9	September	418,343.27	6,734,804.04	285,388.59
22	2,010	10	October	365,252	7,100,056.04	330,510.76
23	2,010	11	November	290,670.36	7,390,726.4	358,088.54
24	2,010	12	December	368,093.98	7,758,820.38	341,338.78

```sql
-- ================================================================================
-- QUESTION 3: Top 3 Products by Profit Within Each CategoryS
-- Identify the top 3 most profitable products in every category using a
-- ranked CTE, correctly handling the fact that window functions cannot be
-- filtered directly in a WHERE clause.
-- ================================================================================
with ranked_products as (
    select
        p.product_category,
        p.product_name,
        sum(oi.profit) as total_profit,
        row_number() over (
            partition by p.product_category
            order by sum(oi.profit) desc
        ) as profit_rank
    from order_items oi
    join products p
        on oi.product_id = p.product_id
    group by p.product_category, p.product_name
)
select
    product_category,
    product_name,
    total_profit,
    profit_rank
from ranked_products
where profit_rank <= 3
order by product_category, profit_rank;
-- Insight: Top performers are dominated by high-ticket equipment
-- (executive chairs, binding machines, copiers/printers) rather than everyday
-- supplies. The Furniture categorys #1 product ($79,284.78) alone outearns
-- its #2 and #3 combined, showing profit within categories is concentrated in
-- a few standout items, not spread evenly.
```

products 1

with ranked_products as (: Enter a SQL expression to filter results (use

Grid

Text

Record

	product_category	product_name	total_profit	profit_rank
1	Furniture	Global Troy™ Executive Leat	79,284.78	1
2	Furniture	Global Leather Executive Ch	14,537.74	2
3	Furniture	SAFCO PlanMaster Heigh-A	14,405.68	3
4	Office Supplies	GBC DocuBind 200 Manual	50,462.53	1
5	Office Supplies	Fellowes PB300 Plastic Com	45,927.85	2
6	Office Supplies	Fellowes PB500 Electric Pun	32,566.56	3
7	Technology	Hewlett Packard LaserJet 33	73,299.61	1
8	Technology	Hewlett-Packard cp1700 [D,	55,281.39	2
9	Technology	Canon PC940 Copier	47,705.85	3

```sql
-- ============================================================================
-- QUESTION 4: Month-over-Month and Year-over-Year Revenue Growth
-- Calculate both MoM and YoY percentage revenue change in a single query,
-- with divide-by-zero protection.
-- ============================================================================
with monthly_revenue as (
    select
        o.year,
        o.month,
        o.month_name,
        sum(oi.sales) as revenue
    from orders o
    join order_items oi
        on o.order_id = oi.order_id
    group by o.year, o.month, o.month_name
)
select
    year,
    month,
    month_name,
    revenue,
    lag(revenue) over (
        order by year, month
    ) as prev_month_revenue,
    case
        when lag(revenue) over (order by year, month) is null then null
        when lag(revenue) over (order by year, month) = 0 then null
        else round(
            (revenue - lag(revenue, 12) over (order by year, month))
            / lag(revenue, 12) over (order by year, month) * 100, 2
        )
    end as yoy_growth_pct
from monthly_revenue
order by year, month;
-- Insight: Monthly revenue is volatile with no consistent growth pattern —
-- YoY swings range from -30% to +46% within 2012 alone. This confirms your
-- earlier finding of no clear multi-year growth trend, and suggests revenue
-- is driven more by individual large orders than steady business expansion.
```

orders 1

with monthly_revenue as (| Enter a SQL expression to filter results (use

Grid | Text | Record

	123 year	123 mor	A-Z month_n	123 reve	123 prev_m	123 yoy_growth_p
1	2,009	1	January	516,302.97	[NULL]	[NULL]
2	2,009	2	February	332,480.63	516,302.97	[NULL]
3	2,009	3	March	411,628.71	332,480.63	[NULL]
4	2,009	4	April	393,276.47	411,628.71	[NULL]
5	2,009	5	May	230,145.54	393,276.47	[NULL]
6	2,009	6	June	263,456.06	230,145.54	[NULL]
7	2,009	7	July	380,503.97	263,456.06	[NULL]
8	2,009	8	August	329,754.74	380,503.97	[NULL]
9	2,009	9	September	325,292.3	329,754.74	[NULL]
10	2,009	10	October	361,555.28	325,292.3	[NULL]
11	2,009	11	November	248,933.45	361,555.28	[NULL]
12	2,009	12	December	415,809.35	248,933.45	[NULL]
13	2,010	1	January	336,526.68	415,809.35	-34.82
14	2,010	2	February	271,580.52	336,526.68	-18.32
15	2,010	3	March	217,808.01	271,580.52	-47.09
16	2,010	4	April	266,968.6	217,808.01	-32.12
17	2,010	5	May	283,534.31	266,968.6	23.2
18	2,010	6	June	293,080.67	283,534.31	11.24
19	2,010	7	July	229,885.5	293,080.67	-39.58
20	2,010	8	August	207,937.01	229,885.5	-36.94
21	2,010	9	September	418,343.27	207,937.01	28.61
22	2,010	10	October	365,252	418,343.27	1.02
23	2,010	11	November	290,670.36	365,252	16.77
24	2,010	12	December	368,093.98	290,670.36	-11.48

Refresh | Save | Cancel | Export data

```sql
-- QUESTION 5: High-Value, High-Risk Customers
-- Identify customers whose total sales rank in the top 20% but whose average
-- profit margin falls below the company-wide average — flagging high-revenue
-- customers who are quietly unprofitable.
-- ================================================================================
with customer_metrics as (
    select
        c.customer_id,
        c.customer_name,
        sum(oi.sales) as total_sales,
        avg(oi.profit_margin) as avg_margin
    from order_items oi
    join orders o
        on o.order_id = oi.order_id
    join customers c
        on c.customer_id = o.customer_id
    group by c.customer_id, c.customer_name
),
sales_threshold as (
    select
        percentile_cont(0.8) within group (
            order by total_sales
        ) as p80_sales
    from customer_metrics
),
company_avg as (
    select avg(profit_margin) as avg_margin
    from order_items
)
select
    cm.customer_name,
    round(cm.total_sales, 2) as total_sales,
    round(cm.avg_margin, 2) as avg_margin
from customer_metrics cm,
     sales_threshold st,
     company_avg ca
where cm.total_sales >= st.p80_sales
    and cm.avg_margin < ca.avg_margin
order by cm.total_sales desc;
-- Insight: All flagged customers show negative average margins despite being
-- top-20% spenders — several severely so (Jay Fine at -82.77%). This confirms
-- high revenue does not guarantee profitability at the customer level, and these
-- accounts need margin review even if their total dollar contribution looks strong.
```

customers 1

with customer_metrics as (| Enter a SQL expression to filter results (use

Grid | Text | Record

	customer_name	total_sales	avg_margin
1	Deborah Brumfield	97,433.14	-29.77
2	Roy Skaria	92,542.16	-18.71
3	Darren Budd	84,606.21	-21.02
4	Sylvia Foulston	83,277.42	-19.97
5	Liz MacKendrick	76,306.43	-19.12
6	Ed Braxton	75,802.76	-35.54
7	Mark Cousins	70,579.73	-17.55
8	Raymond Book	70,426.59	-15.29
9	Jasper Cacioppo	61,350.09	-27.19
10	Noel Staavos	59,092.09	-13.88
11	Carlos Soltero	55,575.09	-16.32
12	Edward Nazzal	54,511.8	-29.81
13	Jack Lebron	51,155.8	-24.05
14	Logan Haushalter	50,662.95	-16.8
15	Harold Dahlen	46,556.16	-16.69
16	Roy Phan	46,546.16	-31.68
17	Ruben Dartt	46,399.44	-32.06
18	Christine Abelman	45,365.75	-25.22
19	Brian DeCherney	44,950.79	-33.33
20	Helen Wasserman	44,416.37	-35.54
21	Nicole Brennan	44,184.61	-33.12
22	Julia West	44,072.81	-20.62
23	Steve Chapman	42,872.99	-22.16
24	Jennifer Patt	42,356	-14.81
25	Doug Bickford	41,159.65	-19.29
26	Sally Matthias	40,273.76	-30.37
27	Charles Crestani	40,030.82	-23.1

Refresh | Save | Cancel

```sql
-- ============================================================================
-- QUESTION 6: Sub-Category Profitability Rank Within Each Region
-- Rank every product sub-category's profit margin within its own region,
-- surfacing each region's best and worst-performing sub-category.
-- ============================================================================
with subcategory_region_margin as (
    select
        c.region,
        p.product_sub_category,
        avg(oi.profit_margin) as avg_margin,
        rank() over (
            partition by c.region
            order by avg(oi.profit_margin) desc
        ) as best_rank,
        rank() over (
            partition by c.region
            order by avg(oi.profit_margin) asc
        ) as worst_rank
    from order_items oi
    join orders o
        on o.order_id = oi.order_id
    join customers c
        on c.customer_id = o.customer_id
    join products p
        on p.product_id = oi.product_id
    group by c.region,
             p.product_sub_category
)
select
    region,
    product_sub_category,
    round(avg_margin, 2) as avg_margin,
    case
        when best_rank = 1 then 'Best in Region'
        when worst_rank =1 then 'worst in Region'
    end as performance_flag
from subcategory_region_margin
where best_rank = 1 or worst_rank = 1
order by region, avg_margin desc;
-- Insight: Labels is the strongest sub-category across every single region
-- (~26-28% margin), while Appliances is the weakest in 3 of 4 regions
-- (down to -59.64% in East). This consistency suggests a company-wide pricing
-- or cost issue with Appliances rather than a regional problem, and Labels could
-- be a category worth expanding.
```

customers(+) 1

with subcategory_region_margin as (select c.region, p.pr | Enter a SQL expression to fi

Grid | Text | Record

	region	product_sub_category	avg_margin	performance_flag
1	Central	Labels	26.97	Best in Region
2	Central	Appliances	-41.77	worst in Region
3	East	Labels	27.03	Best in Region
4	East	Appliances	-59.64	worst in Region
5	South	Labels	25.95	Best in Region
6	South	Copiers and Fax	-43.89	worst in Region
7	West	Labels	27.76	Best in Region
8	West	Appliances	-45.77	worst in Region

Refresh | Save | Cancel | Export data

```sql
-- ============================================================================
-- QUESTION 7: Discount Threshold Where Profitability Breaks Even
-- Group orders into discount bands and identify the exact band where average
-- profit margin flips from positive to negative.
-- ============================================================================
select
    discount_band,
    count(*) as num_orders,
    round(avg(discount), 4) as avg_discount,
    round(avg(profit_margin), 2) as avg_profit_margin
from order_items
group by discount_band
having avg(profit_margin) < 0
order by avg(profit_margin) asc;
-- Insight: All discount bands show negative average margin, even 'No Discount'
-- orders (-7.4%). This confirms discounting is not the root cause of losses —
-- the problem lies elsewhere (e.g., underpricing or high-cost sub-categories
-- like Appliances), not in how much discount is given.
```

order_items 1

```
select discount_band, count(*) as num_orders, round(avg
```

Enter a SQL e

Grid

Text

	discount_banc	num_orders	avg_discount	avg_profit_margir
1	High	2	0.23	-38.86
2	Medium	748	0.1002	-16.33
3	Low	6,893	0.0496	-13.6
4	No Discount	756	0	-7.4

```sql
-- ============================================================================
-- QUESTION 8: Customer Order Frequency & Retention Segments
-- Classify customers into One-Time, Occasional, and Frequent segments based
-- on order count, then measure total profit contribution per segment.
-- ============================================================================
with customer_orders as (
    select
        c.customer_id,
        c.customer_name,
        count(distinct o.order_id) as order_count,
        sum(oi.profit) as total_profit
    from order_items oi
    join orders o
        on o.order_id = oi.order_id
    join customers c
        on c.customer_id = o.customer_id
    group by c.customer_id, c.customer_name
),
segmented_customers as (
    select
        customer_id,
        customer_name,
        order_count,
        total_profit,
        case
            when order_count = 1 then 'One-Time'
            when order_count between 2 and 5 then 'Occasional'
            else 'Frequent'
        end as customer_segment_type
    from customer_orders
)
select
    customer_segment_type,
    count(*) as num_customers,
    round(sum(total_profit), 2) as total_segment_profit,
    round(avg(total_profit), 2) as avg_profit_per_customer
from segmented_customers
group by customer_segment_type
order by total_segment_profit desc;
-- Insight: Frequent customers (467 people, 59% of the base) drive $1.24M in
-- profit — the vast majority of total profit. One-time buyers are a tiny group
-- (10 customers) contributing almost nothing, confirming retention/repeat
-- business matters far more than one-off acquisition here.
-- ============================================================================
```

Results 1

with customer_orders as (select c.customer_id, c.custom | Enter a SQL expression to filt

Grid | Text | Record

	customer_segment_type	num_customers	total_segment_profit	avg_profit_per_custom
1	Frequent	467	1,241,432.32	2,658.31
2	Occasional	317	278,159.84	877.48
3	One-Time	10	2,175.82	217.58

Refresh | Save | Cancel | Export data | 200

3 row(s) fetched - 0.028s, on 2026-08-26 at 01:02:25

```sql
-- QUESTION 9: Shipping Performance vs. Category Benchmark
-- Identify every order that shipped slower than its product category's
-- average shipping time, and rank how far over benchmark each one is.
-- ========================================================================
select
    oi.row_id,
    p.product_category,
    o.shipping_days,
    cat_avg.avg_shipping_days,
    round(o.shipping_days - cat_avg.avg_shipping_days, 2) as days_over_benchmark,
    rank() over (
        partition by p.product_category
        order by o.shipping_days - cat_avg.avg_shipping_days desc
    ) as delay_rank
from order_items oi
join orders o
    on o.order_id = oi.order_id
join products p
    on p.product_id = oi.product_id
join (
    select
        p2.product_category,
        avg(o2.shipping_days) as avg_shipping_days
    from order_items oi2
    join orders o2
        on o2.order_id = oi2.order_id
    join products p2
        on p2.product_id = oi2.product_id
    group by p2.product_category
) cat_avg
    on cat_avg.product_category = p.product_category
where o.shipping_days > cat_avg.avg_shipping_days
order by p.product_category, delay_rank
limit 30;
-- Insight: Furniture average shipping time is just ~2 days, but the worst outlie
-- took 17 days — 15 days over benchmark. A large cluster of orders consistently
-- ships 7-9 days (5-7 days late), suggesting a systemic fulfillment issue for
-- certain Furniture items, not just isolated incidents.
```

order_items(+) 1

select oi.row_id, p.product_category, o.shipping_days, cat | Enter a SQL

Grid | Text | Record

	123 row	A-Z produc	123 shipp	123 avg_ship	123 days_over_b	123 delay_ra
1	2,703	Furniture	17	2.0011600928	15	1
2	4,776	Furniture	9	2.0011600928	7	2
3	6,751	Furniture	9	2.0011600928	7	2
4	3,666	Furniture	9	2.0011600928	7	2
5	2,675	Furniture	9	2.0011600928	7	2
6	2,210	Furniture	9	2.0011600928	7	2
7	6,069	Furniture	9	2.0011600928	7	2
8	7,472	Furniture	9	2.0011600928	7	2
9	392	Furniture	9	2.0011600928	7	2
10	6,068	Furniture	9	2.0011600928	7	2
11	2,082	Furniture	9	2.0011600928	7	2
12	379	Furniture	9	2.0011600928	7	2
13	4,206	Furniture	9	2.0011600928	7	2
14	821	Furniture	7	2.0011600928	5	14
15	1,589	Furniture	7	2.0011600928	5	14
16	6,426	Furniture	7	2.0011600928	5	14
17	4,393	Furniture	7	2.0011600928	5	14
18	6,869	Furniture	7	2.0011600928	5	14
19	3,052	Furniture	7	2.0011600928	5	14
20	3,200	Furniture	7	2.0011600928	5	14
21	648	Furniture	7	2.0011600928	5	14
22	5,947	Furniture	7	2.0011600928	5	14
23	5,838	Furniture	7	2.0011600928	5	14

Refresh | Save | Cancel | Export data

200 | 30 | 30 row(s) fetched - 0.048s (0.001s fetch), on 2026-08-26 at 01:28:25

```sql
-- ============================================================================
-- QUESTION 10: Executive KPI Snapshot with Year-over-Year Comparison
-- A single query producing revenue, profit, margin, and order count for the
-- current year vs. previous year side by side
-- ============================================================================
select
    round(sum(case when o.year = 2012 then oi.sales else 0 end), 2) as revenue_2012,
    round(sum(case when o.year = 2011 then oi.sales else 0 end), 2) as revenue_2011,
    round(sum(case when o.year = 2012 then oi.profit else 0 end), 2) as profit_2012,
    round(sum(case when o.year = 2011 then oi.profit else 0 end), 2) as profit_2011,
    round(
        sum(case when o.year = 2012 then oi.profit else 0 end)
        / nullif(sum(case when o.year = 2012 then oi.sales else 0 end), 0) * 100, 2
    ) as margin_pct_2012,
    round(
        sum(case when o.year = 2011 then oi.profit else 0 end)
        / nullif(sum(case when o.year = 2011 then oi.sales else 0 end), 0) * 100, 2
    ) as margin_pct_2011,
    count(distinct case when o.year = 2012 then o.order_id end) as orders_2012,
    count(distinct case when o.year = 2011 then o.order_id end) as orders_2011
from order_items oi
join orders o
    on o.order_id = oi.order_id
where o.year in (2011, 2012);
-- Insight: 2012 revenue grew 8.2% over 2011 ($3.72M vs $3.44M) and order volume rose
-- too (1,392 vs 1,317), but profit actually declined ($341.9K vs $381.5K) despite
-- margin staying flat at 11.1%. More orders and revenue did not translate to more
-- profit — worth investigating cost structure or product mix shifts in 2012.
```

